JavaScript and Accessibility Review

Common ARIA Accessibility Attributes

- `aria-expanded` **attribute**: Used to convey the state of a toggle (or disclosure) feature to screen reader users.

```
<button id="menuBtn" aria-expanded="false">Menu</button>

<script>
  const btn = document.getElementById("menuBtn");

  btn.addEventListener("click", () => {
    const expanded = btn.getAttribute("aria-expanded") === "true";
    btn.setAttribute("aria-expanded", String(!expanded));
  });
</script>
```

Menu

Open Sandbo

- `aria-haspopup` **attribute**: This state is used to indicate that an interactive element will trigger a pop-up element when activated. Y can only use the `aria-haspopup` attribute when the pop-up has one of the following roles: `menu`, `listbox`, `tree`, `grid`, or `dialog`. The value of `aria-haspopup` must be either one of these roles or `true`, which is the same as `menu`.

```
<button
  id="menubutton"
  aria-haspopup="menu"
  aria-controls="filemenu"
  aria-expanded="false"
>
  File
</button>

<ul
  id="filemenu"
  role="menu"
  aria-labelledby="menubutton"
  hidden
>
  <li role="menuitem" tabindex="-1">Open</li>
```

- Open
- New
- Save
- Delete

Open Sandbo

- `aria-checked` **attribute**: This attribute is used to indicate whether an element is in the checked state. It is most commonly used wh creating custom checkboxes, radio buttons, switches, and listboxes.

```
<div
  id="checkbox"
  role="checkbox"
  aria-checked="true"
  tabindex="0"
  style="
    display: inline-flex;
    align-items: center;
    gap: 6px;
    cursor: pointer;
  "
>
  <span
    id="box"
    aria-hidden="true"
    style="
      width: 16px;
      height: 16px;
      border: 2px solid blue;
      background: blue;
      display: inline-block;
```

Checkbox

Open Sandbo

- `aria-disabled` **attribute**: This state is used to indicate that an element is disabled only to people using assistive technologies, such screen readers.

```
<div
  id="editBtn"
  role="button"
  tabindex="-1"
  aria-disabled="true"
```

```
  const editBtn = document.getElementById("editBtn");
  const toggleBtn = document.getElementById("toggle");

  toggleBtn.addEventListener("click", () => {
    const disabled = editBtn.getAttribute("aria-disabled") === "true";

    editBtn.setAttribute("aria-disabled", String(!disabled));
    editBtn.tabIndex = disabled ? 0 : -1;
```

Edit

Toggle Disabled

Open Sandbo

- **`aria-selected` attribute**: This state is used to indicate that an element is selected. You can use this state on custom controls like a tabbed interface, a listbox, or a grid.

```
<div role="tablist">
  <button role="tab" aria-selected="true">Tab 1</button>
  <button role="tab" aria-selected="false">Tab 2</button>
  <button role="tab" aria-selected="false">Tab 3</button>
</div>

<script>
  const tabs = document.querySelectorAll('[role="tab"]');

  tabs.forEach((tab) => {
    tab.addEventListener("click", () => {
      tabs.forEach(t => t.setAttribute("aria-selected", "false"));
      tab.setAttribute("aria-selected", "true");
    });
  });
</script>
```

Tab 1 Tab 2 Tab 3

Open Sandbo

```
      id="tab1"
      aria-controls="section1"
      aria-selected="true"
    >
      Tab 1
    </button>
    <button
      role="tab"
      id="tab2"
      aria-controls="section2"
      aria-selected="false"
    >
      Tab 2
    </button>
    <button
      role="tab"
      id="tab3"
      aria-controls="section3"
```

- `hidden` **attribute**: Hides inactive panels from both visual and assistive technology users.

Working with Live Regions and Dynamic Content

- `aria-live` **attribute**: Makes part of a webpage a live region, meaning any updates inside that area will be announced by a screen reader so users don't miss important changes.
- `polite` **value**: Most live regions use this value. This value means that the update is not urgent, so the screen reader can wait until it finishes any current announcement or the user completes their current action before announcing the update.

Here is an example of a live region that is dynamically updated by JavaScript:

```
<div aria-live="polite" id="status"></div>

<button id="updateStatus">Update Status</button>

<script>
  const statusEl = document.getElementById("status");
  const btn = document.getElementById("updateStatus");

  btn.addEventListener("click", () => {
    statusEl.textContent = "Your file has been successfully uploaded.";
  });
</script>
```

Open Sandbo

- **`contenteditable` attribute**: Turns the element into a live editor, allowing users to update its content as if it were a text field. Wher there is no visible label or heading for a contenteditable region, add an accessible name using the `aria-label` attribute to help scr reader users understand the purpose of the editable area.

```
<div
  contenteditable="true"
  aria-label="Note editor"
  id="editor"
  style="border: 1px solid #ccc; padding: 8px;"
>
  Editable content goes here
</div>

<p id="status" aria-live="polite"></p>

<script>
  const editor = document.getElementById("editor");
  const status = document.getElementById("status");

  editor.addEventListener("input", () => {
    status.textContent = "Content updated";
  });
</script>
```

Editable content goes here

Open Sandbo

`focus` and `blur` Events

- **`blur` event**: Fires when an element loses focus.

```
<input
  id="nameInput"
  type="text"
  placeholder="Type here and click outside"
  aria-label="Name input"
/>

<p id="status" aria-live="polite"></p>
```

```
  });
</script>
```

Type here and click outside

Open Sandbo

- **`focus` event**: Fires when an element receives focus.

```
<input
  id="emailInput"
  type="email"
  placeholder="Click or tab into this field"
  aria-label="Email input"
/>

<p id="status" aria-live="polite"></p>

<script>
  const input = document.getElementById("emailInput");
  const status = document.getElementById("status");

  input.addEventListener("focus", () => {
    status.textContent = "Input received focus";
  });
</script>
```

Click or tab into this field

Open Sandbo

Assignment

Ask for Help